

2026 전기

Physical AI 졸업과제 착수 보고서

LeRobot SO-101을 활용한 지능형 로봇팔 제어

팀명

키자니아 패밀리

팀원

학번	이름	이메일
202355561	여채언	codjs2659@pusan.ac.kr
202355536	문여원	myw0422@pusan.ac.kr
202355562	옥소미	osm0071@pusan.ac.kr

Contents

1	개요	2
1.1	과제 배경	2
1.1.1	Physical AI	2
1.1.2	Physical AI 졸업과제	2
1.2	과제 목표	2
1.2.1	1차 미션	2
1.2.2	2차 미션	2
1.2.3	최종 미션	2
1.3	기대 효과	2
2	과제 구성	3
2.1	문제 정의 및 요구조건 분석	3
2.1.1	대상 문제 정의	3
2.1.2	큐브 조건	3
2.1.3	경기장 및 지정영역 조건	4
2.1.4	시연 및 평가 환경	4
2.1.5	평가기준	4
2.1.6	설계시 고려사항	4
2.2	시스템 설계 및 적용 기술	5
2.2.1	시스템 구성	5
2.2.2	개발 환경	5
2.2.3	데이터 수집 및 단계별 정책 학습 전략	6
2.2.3.1	Tele-operation 기반 데이터 수집 및 SmolVLA Fine-tuning	6
2.2.3.2	Simulation 기반 데이터 확장	6
2.2.3.3	Reinforcement Learning 기반 정책 최적화	6
2.3	현실적 제약 사항 및 대응 방안	7
2.3.1	Orin Nano PyTorch CUDA 추론 환경과 LeRobot 제어 환경 충돌	7
2.3.1.1	문제	7
2.3.1.2	검토한 대응 방안	7
2.3.1.3	채택한 방안 : 방안 3 - LeRobot Async Inference	8
2.3.2	관절 유격감 문제	8
2.3.2.1	문제	9
2.3.2.2	대응	9
3	개발 일정 및 역할	9
3.1	추진 체계 및 일정	9
3.2	구성원 역할 분담	10
4	참고 문헌	10

1 개요

1.1 과제 배경

1.1.1 Physical AI

Physical AI는 로봇이나 자율주행차처럼 실제 환경에서 움직이는 시스템이 주변을 인식하고 이해한 뒤, 상황을 판단해 행동하는 기술이다.

기존의 로봇이 정해진 동작을 반복하는 자동화 장비에 가까웠다면, Physical AI 기반 로봇은 주변 환경을 파악하고 상황에 맞게 스스로 동작을 조정하는 방향으로 발전하고 있다. 이미지, 영상, 센서 데이터 등을 통해 물체가 어디에 있고 어떻게 움직일 수 있는지 파악하고, 이를 로봇이 실제로 수행할 수 있는 동작으로 연결한다.

1.1.2 Physical AI 졸업과제

본 졸업과제는 이러한 Physical AI의 개념을 실제 로봇 팔 제어에 적용해 보는 과제이다. Physical AI 졸업과제의 종목 2, "LeRobot SO-101을 활용한 지능형 로봇 팔 제어"는 로봇 팔을 이용해 테이블 위에 무작위로 놓인 블록을 집고, 지정된 영역으로 옮긴 뒤 안정적으로 쌓아 올리는 것을 목표로 한다.

이 과제에서는 단순히 로봇 팔을 움직이는 것뿐만 아니라, 블록의 위치를 확인하고 집을 순서를 정한 뒤 정확한 위치에 내려놓아야 한다. 따라서 물체 인식, 동작 판단, 로봇 팔 제어가 모두 필요한 과제라고 볼 수 있다.

1.2 과제 목표

1.2.1 1차 미션

본 과제의 1차 미션은 LeRobot SO-101 로봇 팔이 전체 경기장 안에 무작위로 배치된 5개의 블록을 카메라로 인식하고, 각 블록을 집어 지정 영역인 20cm × 10cm 공간 안으로 옮기는 것이다. 이를 위해 카메라를 이용해 블록의 위치를 파악하고, 로봇 팔이 블록을 집어 옮길 수 있도록 이동 경로와 그리퍼 동작을 제어한다.

1.2.2 2차 미션

2차 미션은 지정 영역으로 옮긴 블록을 탑처럼 위로 쌓아 올리는 것이다. 블록을 안정적으로 쌓기 위해서는 놓는 위치가 정확해야 하고, 쌓인 블록이 무너지지 않도록 균형도 고려해야 한다.

1.2.3 최종 미션

최종적으로는 5개의 블록 중 가능한 많은 블록을 정확하게 옮기고 안정적으로 쌓아 올리고, 불필요한 동작을 줄여 수행 시간을 단축하는 것을 목표로 한다.

1.3 기대 효과

본 과제를 통해 실제 로봇 팔을 이용한 물체 인식과 행동 결정 과정을 이해할 수 있다. 카메라로 블록의 위치를 파악하고, 로봇 팔을 움직여 물체를 집고 옮기는 과정을 구현하면서 인식 결과가 실제 동작으로 이어지는 과정을 이해할 수 있다. 또한 블록의 위치가 매번 달라지는 환경에서 작업을 수행하기 때문에,

단순히 정해진 동작을 반복하는 방식이 아니라 상황에 따라 적절한 행동을 선택하는 정책 학습 과정을 이해할 수 있다. Tele-operation으로 수집한 expert demonstration 데이터를 활용한 모방학습을 통해 기본적인 Pick & Place 및 Pick & Stack 동작을 학습하고, 이후 강화학습을 통해 블록 배치 정확도, 적재 성공 여부, 수행 시간 등을 보상 기준으로 반영하여 정책을 개선할 수 있다. 이를 통해 로봇이 다양한 초기 조건에서도 안정적으로 과제를 수행하도록 학습 과정을 설계하는 방법을 익힐 수 있다. 본 과제는

향후 다양한 물체 조작 과제나 실제 서비스 로봇 응용으로 확장될 수 있다. 블록 이동 및 적재 과제에서 확보한 인식, 정책 학습, 로봇 제어 기술은 물류 자동화, 정리 정돈, 조립 보조, 교육용 로봇 실습 등 여러 분야에 활용될 수 있으며, 실제 로봇 환경에서 VLA 기반 정책 모델을 적용하는 기초 연구로서 의미를 가진다.

2 과제 구성

2.1 문제 정의 및 요구조건 분석

2.1.1 대상 문제 정의

구분	내용
과제명	LeRobot SO-101을 활용한 Pick & Stack
핵심 목표	mission 1: 무작위로 배치된 블록 5개를 인식하여 지정 영역에 옮기기 mission 2: 5개의 블록을 위로 탑 처럼 적재
사용 장비	LeRobot SO-101 로봇 팔(follower arm), Teleoperation 용 leader arm, 카메라 3대, 추론용 NVIDIA Jetson Orin Nano, 학습용 NVIDIA RTX 4090 PC
문제 성격	비전 기반 상태 인식 + 정책 학습 기반 행동 생성 + 로봇 팔 제어
제한 시간	총 5분 이내 두가지 미션을 수행

2.1.2 큐브 조건

항목	요구 조건
개수	총 5개
크기	5cm × 5cm × 2cm
형태	직육면체
초기 상태	모든 블록은 바닥에 놓여 있어야 하며, 서로 겹치지 않아야 함
바닥 접촉면	넓은 면인 5cm × 5cm 면이 바닥을 향하도록 배치
색상/재질/무게	팀별 자유 선택 가능

2.1.3 경기장 및 지정영역 조건

항목	요구 조건
경기장	A3 용지 (420mm * 297mm)
지정 영역 크기	20cm × 10cm
경계선 두께	10mm 20mm
지정 영역 위치	A3 용지 기준 가로 중앙, 세로 상단 부근
영역 판정	블록이 경계선에 걸쳐도 지정 영역을 벗어나지 않으면 IN으로 인정
로봇 배치	경기장 밖에 설치 가능하나, 블록에 닿을 수 있는 범위 내 배치 필요

2.1.4 시연 및 평가 환경

항목	내용
환경 준비	각 팀이 로봇, 카메라, 개발보드 등을 직접 준비
로봇 사용	팀별로 LeRobot SO-101 로봇을 직접 세팅하여 사용
카메라 설정	높이와 각도는 팀별 자율 설정 가능
제어 방식	규칙 기반 제어, 비전 기반 제어, 모방학습 등 제한 없음
발표 내용	사용한 제어 기법과 해당 기법의 효과성, 특징, 우수성 설명 필요

2.1.5 평가기준

평가 항목	내용
제한 시간 초과 시	5분 내 적재한 블록 수 기준으로 점수 산정
제한 시간 내 목표 달성 시	5개 전체 적재 완료 시 소요 시간 기준 보너스 점수 부여
성공 인정 기준	블록이 5초 이상 안정적으로 유지될 경우 성공 인정
평가 요소	수행 시간, 목표 달성도, 적재 안정성, 제어 방법의 타당성

2.1.6 설계시 고려사항

고려사항	반영 내용	필요성
블록 재질 선정	그리퍼와의 마찰력을 높일 수 있는 재질의 블록 사용	파지 중 블록이 미끄러지거나 떨어지는 문제를 줄이고 안정적인 집기 동작을 수행하기 위함
경기장 재질 선정	빛 반사를 줄이기 위해 무광 재질의 경기장 사용	조명 반사로 인한 인식 오류를 줄이고 안정적인 비전 인식을 수행하기 위함
지정 영역 표시	A3 경기장 색상과 대비가 큰 검은색 라인을 사용하여 지정 영역 표시	카메라 영상에서 지정 영역의 경계를 명확히 인식하고, 블록 배치 성공 여부를 판단하기 위함
블록 색상 선정	블록 색상을 빨강, 노랑, 초록, 파랑, 보라색으로 구성	각 블록이 경기장 바닥 및 지정 영역 라인과 시각적으로 명확히 구분되도록 하여, 카메라 기반 인식 과정에서 색상 분류와 위치 검출이 용이하도록 하기 위함
실패 대응	파지 실패, 블록 미끄러짐, 적재 실패 발생 시 재시도 또는 다음 블록 처리 전략 고려	제한 시간 내 최대한 많은 블록을 성공적으로 옮기고 적재하기 위함

2.2 시스템 설계 및 적용 기술

본 과제는 LeRobot SO-101 로봇 팔을 활용하여 테이블 위에 무작위로 배치된 5개의 블록을 인식하고, 이를 지정 영역으로 옮기거나 순차적으로 적재하는 Pick & Stack 문제를 대상으로 한다.

2.2.1 시스템 구성

본 과제의 시스템은 LeRobot SO-101 로봇 팔이 카메라로 작업 환경을 관측하고, SmolVLA 기반 정책 모델이 관측 정보를 바탕으로 다음 행동을 추론한 뒤, 이를 로봇 관절 제어 명령으로 변환하여 Pick & Place 및 Pick & Stack 동작을 수행하는 구조로 설계한다.

순서	구성 요소	주요 내용	역할
1	작업 환경 및 입력 장치	블록, 목표 영역, 카메라	테이블 위 블록 배치와 목표 영역을 관측하기 위한 환경 구성
2	로봇 플랫폼	LeRobot SO-101, 로봇 제어용 컴퓨터	실제 블록 집기, 이동, 배치, 적재 동작 수행
3	관측 데이터 처리	카메라 영상 입력, 블록 및 목표 영역 정보 추출	현재 작업 상태를 모델 입력으로 사용할 수 있는 형태로 구성
4	정책 모델	SmolVLA fine-tuning 모델	관측 정보를 바탕으로 다음 로봇 행동을 추론
5	로봇 제어 및 실행	로봇 관절 제어 명령 변환, 동작 실행	모델이 추론한 행동을 실제 로봇 동작으로 수행
6	성능 평가 및 분석	수행 결과, 성공 여부, 실패 유형	미션별 성공률, 수행 시간, 안정성 등을 분석하여 방법론별 성능 비교에 활용

2.2.2 개발 환경

본 과제에서는 실제 로봇 실험과 학습을 연계하기 위해 다음과 같은 개발 환경을 활용한다.

구분	사용 도구 및 환경	활용 목적
로봇 플랫폼	LeRobot SO-101	실제 로봇 팔 제어 및 real-world 데이터 수집
학습 모델	SmolVLA	Vision-Language-Action 기반 행동 정책 학습
실제 데이터 수집	Tele-operation	사람이 직접 조작한 expert demonstration episode 수집
시뮬레이션 데이터 수집	Isaac Sim	다양한 초기 조건과 블록 배치에 대한 simulation episode 수집
학습 방식	Fine-tuning, simulation 기반 학습, reinforcement learning	실제 데이터 및 시뮬레이션 데이터를 활용한 단계별 정책 학습 및 성능 개선
입력 장치	카메라	블록, 목표 영역, 작업 공간 관측
학습 및 추론 장치	NVIDIA RTX 4090	SmolVLA fine-tuning 및 모델 추론 수행
로봇 제어용 컴퓨터	NVIDIA Jetson Orin Nano	LeRobot SO-101과 연결되어 로봇 제어 명령 전달 및 실행

본 과제에서는 SmolVLA의 fine-tuning과 모델 추론을 NVIDIA RTX 4090 PC에서 수행하고, LeRobot SO-101과 직접 연결된 NVIDIA Jetson Orin Nano는 로봇 제어 명령 전달 및 실행을 담당한다. 이를 통해 고성능 연산이 필요한 모델 학습 및 추론 과정과 실제 로봇 제어 과정을 분리하여 시스템을 구성한다. 초기 단계에서는 tele-operation을 통해 실제 로봇의 expert demonstration episode를 수집하고,

이를 활용하여 SmolVLA를 과제 환경에 맞게 fine-tuning한다. 이후 simulation 환경에서 다양한 초기 조건의 데이터를 추가로 확보하고, 마지막 단계에서는 강화학습을 적용하여 정책의 성공률과 안정성을 개선한다.

2.2.3 데이터 수집 및 단계별 정책 학습 전략

2.2.3.1 Tele-operation 기반 데이터 수집 및 SmolVLA Fine-tuning

초기 단계에서는 사람이 직접 LeRobot SO-101을 조작하여 Pick & Place 및 Pick & Stack 동작 데이터를 수집한다. 이때 수집되는 tele-operation episode는 SmolVLA의 Vision-Language-Action 구조에 맞추어, 카메라 기반 관측 정보인 **visual observation**, 과제 수행 조건을 나타내는 **language instruction**, 로봇의 실제 조작 궤적인 **action trajectory**를 포함하도록 구성한다. 수집된 episode는 이후 SmolVLA를 과제 환경에 맞게 fine-tuning하는 데 활용한다.

- 실제 로봇 환경에서 expert demonstration episode를 수집한다.
- 수집된 episode를 **visual observation-language instruction-action trajectory** 형식으로 정리한다.
- 정리된 데이터를 기반으로 SmolVLA를 task-specific하게 fine-tuning한다.
- 초기에는 미션 1에 해당하는 지정 영역 이동 과제를 중심으로 기본 조작 정책을 확보한다.
- 이후 미션 2에 해당하는 블록 순차 적재 과제로 확장하여 난이도를 높인다.
- 학습된 정책은 실제 로봇 환경에서 추론 및 성능 평가를 수행한다.

2.2.3.2 Simulation 기반 데이터 확장

Tele-operation 기반 데이터는 실제 로봇 환경을 반영한다는 장점이 있지만, 사람이 직접 조작하여 수집해야 하므로 데이터 확보에 많은 시간이 필요하고 다양한 상황을 충분히 포함하기 어렵다. 이를 보완하기 위해 simulation 환경을 구축하고, 다양한 초기 조건에서 추가 episode를 생성하여 학습 데이터의 양과 다양성을 확장한다.

Simulation 환경에서는 블록의 초기 위치, 방향, 배치 순서, 목표 영역과의 거리 등을 다양하게 설정한다. 또한 실제 실험에서 반복적으로 수집하기 어려운 예외 상황과 실패 케이스를 포함하여, tele-operation 데이터만으로는 부족한 상황 다양성을 보완한다.

- 블록의 초기 위치, 방향, 배치 순서 등을 다양하게 설정한다.
- 목표 영역과의 거리, 블록 간 간격 등 작업 조건을 변화시킨다.
- 실제 환경에서 반복 수집하기 어려운 실패 상황과 예외 케이스를 생성한다.
- Simulation 데이터 기반 모델과 실제 데이터 기반 모델의 성능을 비교한다.
- 실제 데이터와 simulation 데이터를 함께 활용했을 때의 성능 향상 여부를 검토한다.

Simulation 기반 접근은 대량의 episode를 효율적으로 확보할 수 있다는 점에서 유용하다. 특히 Pick & Stack 과제는 블록 배치 경우의 수가 다양하고, 적재 과정에서 위치 오차가 누적될 수 있으므로 다양한 조건을 학습하는 것이 중요하다. 따라서 simulation 데이터는 정책이 새로운 블록 배치나 초기 조건에서도 안정적으로 동작하도록 일반화 성능을 보완하는 역할을 한다.

다만 simulation 환경은 실제 로봇의 관절 유격, 마찰, 카메라 노이즈, 케이블 간섭, 부품 오차 등을 완전히 반영하기 어렵다. 따라서 simulation 데이터는 실제 데이터를 대체하기보다는 보완하는 용도로 활용하며, 최종적으로는 실제 LeRobot SO-101 환경에서 성능을 검증한다.

2.2.3.3 Reinforcement Learning 기반 정책 최적화

마지막 단계에서는 tele-operation 및 simulation 기반 학습으로 확보한 초기 정책을 강화학습을 통해 추가적으로 최적화한다. 모방학습은 expert demonstration을 따라 하는 데 효과적이지만, 사람이 수집한 데이터에 포함되지 않은 상태나 장기적인 task 성공률을 직접 최적화하는 데에는 한계가 있다. 이를 보완하기 위해 강화학습을 적용하여 평가 기준과 연계된 reward를 설계하고, 정책의 성공률과 안정성을 개선한다.

강화학습 단계에서는 SmolVLA fine-tuning을 통해 확보한 정책을 초기 정책으로 활용하고, policy rollout을 통해 과제 수행 결과를 평가한다. 이후 블록 위치 정확도, 적재 안정성, 수행 시간, 실패 여부 등을 기준으로 reward와 penalty를 부여하여 정책을 점진적으로 개선한다.

- SmolVLA fine-tuning으로 확보한 정책을 초기 정책으로 활용한다.
- 블록을 목표 영역에 정확히 배치할수록 reward를 부여한다.
- 미션 2에서는 블록이 안정적으로 쌓이고 일정 시간 유지될 경우 추가 reward를 부여한다.
- 블록이 쓰러지거나 목표 영역을 벗어나는 경우 penalty를 부여한다.
- 수행 시간이 짧을수록 높은 보상을 부여하여 효율적인 동작을 유도한다.

강화학습을 적용하면 단순히 expert demonstration을 모방하는 수준을 넘어, 미션 성공률, 수행 시간, 블록 위치 정확도, stack 안정성 등 평가 기준과 직접 연결된 목표를 중심으로 정책을 최적화할 수 있다. 특히 미션 2의 블록 적재 과제에서는 집기 성공 여부뿐 아니라 놓는 위치의 정밀도와 적재 후 안정성이 중요하므로, 강화학습을 통해 최종 정책의 안정성과 성공률을 높이고자 한다.

2.3 현실적 제약 사항 및 대응 방안

2.3.1 Orin Nano PyTorch CUDA 추론 환경과 LeRobot 제어 환경 충돌

2.3.1.1 문제

- GPU를 활용한 policy 추론을 수행하기 위해서는 Python 3.10 기반 PyTorch CUDA 환경이 필요하지만, 최신 LeRobot API를 활용한 로봇 제어는 Python 3.12 환경에서 안정적으로 동작한다.

구분	요구 Python 버전	비고
Orin Nano GPU 추론 환경	Python 3.10	NVIDIA 제공 CUDA PyTorch 휠 호환
LeRobot 제어 환경	Python 3.12	최신 LeRobot API/CLI 동작

핵심 충돌은 현재 사용 중인 `lerobot-record`, `lerobot-eval` API가 하나의 Python 프로세스 안에서 observation 수집 → policy 전처리 → PyTorch forward → action 후처리 → 로봇 제어 → 데이터 기록을 모두 수행한다는 점이다.

이렇게 LeRobot CLI는 추론과 제어가 하나의 실행 흐름 안에 통합되어 있어, 별도 코드 작성 없이 명령어 실행만으로 observation 수집, policy 추론, action 실행, 데이터 기록을 자동으로 수행할 수 있다는 장점이 있다.

그러나 Orin Nano 환경에서는 GPU 기반 PyTorch 추론을 위해 Python 3.10 환경이 필요하고, 최신 LeRobot 기반 로봇 제어는 Python 3.12 환경에서 동작한다. 즉, 기존 CLI 구조를 사용하려면 단일 Python 환경에서 **GPU 추론**과 **LeRobot 제어**를 동시에 만족해야 하지만, Python 버전 제약으로 인해 이를 그대로 적용하기 어렵다.

따라서 기존 CLI 기반 실행 방식을 그대로 사용하기에는 제약이 있으며, 정책 추론 프로세스와 로봇 제어 프로세스를 분리하는 절차가 필요하다.

2.3.1.2 검토한 대응 방안

방안 1. ROS2 기반 외부 서버 추론

- Orin Nano는 카메라/observation 송신 및 action 수신·실행 역할만 담당한다.
- 메인 서버(RTX 4090 PC)에서 정책 추론을 수행하고 action chunk를 Orin Nano로 전송한다.
- 한계
 - 대회 환경에서 외부 서버까지의 네트워크 통신 가능 여부가 불확실하다.

- LeRobot CLI를 그대로 사용할 수 없다. 기존 LeRobot CLI는 추론과 제어가 한 프로세스 안에서 함께 동작하는 일체형 루프이므로, 추론을 외부 서버로 옮기려면 policy 호출부를 별도 노드로 분리하고 ROS2 토픽으로 obs/action을 주고받도록 루프를 재구성해야 한다.
- Orin Nano의 GPU 자원을 사용하지 못해 하드웨어 활용도가 떨어진다.
- ROS2 메시지/토픽 설계 등 별도 통신 인프라 구축 부담이 있다.

방안 2. Orin Nano 내부 듀얼 프로세스

- Orin Nano 안에서 환경을 분리하여 3.10 환경에서 추론하고, 3.12 환경에서 제어한다.
- 같은 Orin Nano 안에서 Python 3.10(추론), 3.12(제어) 두 프로세스를 운용하고 ZeroMQ로 obs/action만 주고받는다.
- 한계
 - 기존 LeRobot CLI는 추론/제어 일체형 루프이므로 그대로 적용이 불가능하다. policy 호출부를 별도 프로세스로 분리하는 구조 변경이 필수이다.
 - 동기식 요청-응답 구조이므로 추론 지연이 그대로 제어 루프 지연으로 전파된다.

방안 3. LeRobot Async Inference (RobotClient-PolicyServer 분리)

- LeRobot이 SmolVLA와 함께 공식 제공하는 비동기 추론 프레임워크로, action 예측과 실행을 구조적으로 분리한다.

역할	장비	담당 작업
RobotClient	Jetson Orin Nano	로봇/카메라 I/O, observation 송신, action 실행
PolicyServer	RTX 4090 PC	SmolVLA/ACT 등 policy 모델 추론

- 동작 원리
 - 동기식 추론은 정책이 다음 chunk를 계산하는 동안 로봇이 idle 상태로 대기 → 실시간 lag 발생
 - 비동기 추론은 현재 chunk가 소진되기 전에 다음 chunk를 미리 계산하여 idle frame을 제거하고, 겹치는 구간은 aggregation 함수(예: `weighted_average`)로 병합하여 더 타이트한 제어 루프를 구성한다.
- 장점
 - LeRobot 공식 지원이므로 CLI 루프 재구성을 직접 구현할 필요가 없고 SmolVLA · ACT 등 모든 LeRobot 정책과 호환된다.
 - 추론 지연이 제어 루프를 막지 않는다.
- 한계
 - 방안 1과 동일하게 외부 서버까지의 네트워크 통신이 전제된다.
 - 대역폭 · 지연 변동이 큐 안정성에 영향을 주므로 `actions_per_chunk`, `chunk_size_threshold`, `fps` 튜닝이 필요하다.

2.3.1.3 채택한 방안 : 방안 3 - LeRobot Async Inference

- 공식 프레임워크를 활용하여 구현 부담을 최소화하면서 추론/제어 분리 요구를 충족한다.
- Orin Nano는 RobotClient로 동작하므로 Python 3.12 + LeRobot 환경만 구성하면 되고, GPU 추론 환경(3.10) 충돌 자체가 발생하지 않음

2.3.2 관절 유격감 문제

2.3.2.1 문제

- 팔로워암 동작 중 관절부에 유격이 발생하여 end-effector 위치 정확도가 저하되는 현상이 관찰되었다.

2.3.2.2 대응

- 관절 나사를 조여 유격을 제거함으로써 해결하였다. 동작을 반복하면 불가피하게 관절 나사가 풀어지기 마련이기 때문에, 주기적으로 조여주는 것이 필요하다.

3 개발 일정 및 역할

3.1 추진 체계 및 일정

기간	목표	주요 수행 내용	산출물
5월	Tele-operation 기반 1차 미션 구현 및 검증	• LeRobot SO-101 환경 구축 및 1차 미션 과제 정의 • Tele-operation을 통한 1차 미션(pick and place) episode 데이터 수집 • 수집 데이터 정제 및 학습 데이터셋 구성 • 사전학습 모델(SmolVLA) 기반 제어 모델 Fine-tuning • 추론 실험 및 초기 성능 평가	• 1차 미션 tele-operation dataset • Fine-tuned imitation policy • 초기 성공률 및 실패 사례 분석
6월	Tele-operation 기반 2차 미션 구현 및 검증	• 2차 미션 과제 정의 • Tele-operation을 통한 2차 미션(pick and stack) episode 데이터 수집 • 수집 데이터 정제 및 학습 데이터셋 구성 • 사전학습 모델(SmolVLA) 기반 제어 모델 Fine-tuning • 추론 실험 및 초기 성능 평가	• 2차 미션 tele-operation dataset • Fine-tuned imitation policy • 초기 성공률 및 실패 사례 분석
7월	Simulation 기반 성능 개선	• simulation 환경 구성 • simulation 기반 대량 episode 데이터 수집 • 다양한 블럭 배치, 조명/시점 조건 반영 • Sim 데이터 기반 제어 모델 Fine-tuning • Tele-operation 데이터 기반 모델과 성능 비교 • Sim 기반 모델의 실제 환경 적용 가능성 검토	• Simulation dataset • Sim-trained / Sim+Real fine-tuned policy • Sim-to-Real 성능 분석

기간	목표	주요 수행 내용	산출물
8월	강화학습 기반 정책 최적화	• 모방학습 정책을 초기 정책으로 활용 • 강화학습 reward 설계 • fine-tuning 수행 • 성공률, episode length, 안정성 기준으로 정책 최적화 • 실패 케이스에 대한 보상 구조 및 정책 개선	• RL-trained policy • Reward 설계 문서 • Tele-operation vs sim vs RL 성능 비교
9월	안정화 및 최적화, 문서화	• 전체 방법론별 성능 지표 비교 • 성공률, 수행 시간, 안정성, 일반화 성능 평가 • 로봇 동작 안정화 및 추론 속도 최적화 • 실험 결과 정리 및 문서화 • 최종 보고서 및 발표자료 작성	• 최종 성능 평가표 • 방법론별 장단점 분석 • 최종 보고서 및 발표자료

3.2 구성원 역할 분담

- 여채언
 - 수집된 데이터를 활용한 SmolVLA Fine-tuning 및 하이퍼파라미터 최적화
 - 강화학습(RL) 단계에서의 보상 함수(Reward Function) 설계
 - 모델 추론 성능 최적화
 - 실패 유형 분석 후 정책 개선 및 알고리즘 업데이트
- 옥소미
 - LeRobot SO-101 하드웨어 셋업 및 카메라(Vision), 관절 캘리브레이션
 - Tele-operation 장치를 활용한 실제 환경에서의 전문가 시연 데이터 수집
 - Jetson Orin Nano와 RTX 4090 간의 통신 파이프라인 구축 및 실시간 제어 명령 실행
 - 실제 로봇 환경에서의 미션(Pick & Place & Stack) 성공률 및 안정성 지표 기록/분석
- 문여원
 - Isaac Sim 내 로봇 팔 및 블록 5개에 대한 simulation 환경 구축
 - 무작위 블록 배치, 예외 상황 등 시뮬레이션 기반 대량 에피소드 데이터 생성
 - 실제 데이터와 시뮬레이션 데이터를 SmolVLA 규격(VLA)에 맞춰 통합 및 전처리
 - Sim-to-Real 간극을 줄이기 위한 Domain Randomization(노이즈, 텍스처 변화 등) 적용

4 참고 문헌

1. NVIDIA. "Generative Physical AI." NVIDIA Glossary. <https://www.nvidia.com/en-us/glossary/generative-physical-ai/>
2. Hugging Face. "LeRobot: smolVLA." Hugging Face Docs. <https://huggingface.co/docs/lerobot/smolvla>
3. Hugging Face. "LeRobot: Async Inference." Hugging Face Docs. <https://huggingface.co/docs/lerobot/async>
4. NVIDIA Developer Forums. "Jetson Orin Nano PyTorch 버전 정보." <https://forums.developer.nvidia.com/t/pytorch-for-jetson/72048>

5. NVIDIA. "Jetson Platform용 PyTorch wheel 파일 설치." NVIDIA Deep Learning Documentation.
<https://docs.nvidia.com/deeplearning/frameworks/install-pytorch-jetson-platform/index.html>